

UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
DEPARTAMENTO DE COMPUTACIÓN

PRÁCTICA DE LABORATORIO N° 3

Sensor de luminosidad y Controlador de tensión arterial

Autores:

Genesis V. Cabello M.
C.I:30.367.386

Maivet S. Pereira M.
C.I: 30.814.708

Profesor:

Jose Canache

Asignatura:

Arquitectura del Computador

Objetivo:

Familiarizar al estudiante con la sintaxis del lenguaje ensamblador MIPS32 y su ejecución en el entorno MARS, resolviendo problemas clásicos de programación, prestando especial atención a operaciones de entrada/salida (E/S) mediante el manejo de dispositivos de hardware mapeados en memoria.

1. Investigación Preliminar

1.1. Explique cómo se organiza la memoria cuando un sistema utiliza memory-mapped I/O. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones lw y sw?

En un sistema con Memory-Mapped I/O (MMIO), el espacio de direcciones físicas del procesador se comparte entre la memoria principal (RAM) y los registros de control, estado y datos de los dispositivos periféricos. No existe un espacio de direcciones aislado para la entrada/salida.

Organización y Región de Memoria:

En la arquitectura MIPS32, el espacio de direcciones reservado para los periféricos e interfaces de E/S suele ubicarse en la parte superior del mapa de memoria (por ejemplo, en el segmento Kernel kseg1 a partir de 0x80000000 / 0xA0000000 en hardware real, o en la dirección 0xFFFF0000 dentro del entorno de simulación MARS/SPIM). Esta región se marca intencionalmente como no cacheable para garantizar que cada lectura o escritura acceda directamente al hardware y no a una copia guardada en la memoria caché.

Implicaciones para lw y sw:

No se requieren instrucciones especiales para realizar operaciones de E/S. Las mismas instrucciones de carga y almacenamiento utilizadas para la RAM (lw para leer registros de estado/datos y sw para escribir comandos/controles) son las que interactúan con los periféricos. La única precaución para el programador es asegurarse de acceder a las direcciones base exactas donde están mapeados los registros del dispositivo.

1.2. ¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O?

Diferencia Principal:

- MMIO: Utiliza el mismo bus de datos/direcciones y el mismo espacio de direccionamiento para la RAM y los periféricos.
- E/S por Puertos (Port-Mapped I/O o Isolated I/O): Posee un espacio de direcciones separado e independiente exclusivo para los dispositivos de E/S, al cual se accede mediante instrucciones de máquina dedicadas (por ejemplo, IN y OUT en la arquitectura x86).

Tipo	Ventajas	Desventajas
MMIO	Simplifica el diseño de la CPU y el conjunto de instrucciones (ISA); permite aplicar todos los modos de direccionamiento y operaciones lógicas/aritméticas de la CPU directamente sobre los registros del dispositivo.	Consume espacio en el mapa de memoria RAM disponible y requiere mecanismos de protección de memoria para evitar accesos no autorizados.
E/S por Puertos	No resta espacio al mapa de memoria RAM principal.	Requiere circuitería adicional en la CPU y un repertorio de instrucciones más complejo.

¿Por qué MIPS32 utiliza MMIO?

MIPS32 sigue la filosofía RISC (Reduced Instruction Set Computer), cuyo objetivo es mantener la arquitectura de la CPU y el repertorio de instrucciones lo más simple, uniforme y reducido posible. Utilizar MMIO evita tener que agregar instrucciones especializadas de E/S a la ISA.

1.3. En un sistema con memory-mapped I/O: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?

Problemas de Solapamiento:

Si dos periféricos comparten o solapan sus direcciones de memoria, se produce un conflicto en el bus de datos (bus contention). Si el procesador intenta leer una dirección solapada, ambos dispositivos responderán intentando colocar sus datos en el bus simultáneamente, lo que puede causar corrupción de datos, lecturas erróneas o incluso daños físicos por cortocircuitos en las líneas del bus. En escrituras, ambos dispositivos recibirían el comando, ejecutando acciones no deseadas.

Cómo se evita:

Se evita mediante el diseño del Decodificador de Direcciones (Address Decoder) a nivel de hardware. El decodificador analiza las líneas del bus de direcciones y activa de forma mutuamente excluyente la línea de selección de chip (Chip Select - CS) de un único periférico a la vez. A nivel del sistema operativo, el mapa de memoria se gestiona de manera estricta mediante tablas de asignación de recursos.

1.4. ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

Simplificación del conjunto de instrucciones:

Permite reutilizar la infraestructura existente de la CPU (decodificador de instrucciones, unidad de control, pipeline y los ciclos de ejecuciones de carga y almacenamiento). La CPU no necesita reconocer cuándo está interactuando con un sensor o con la RAM; simplemente ejecuta una lectura o escritura en una dirección del bus.

Instrucciones adicionales para E/S por puertos:

Si se utilizara E/S por puertos, la ISA de MIPS32 tendría que incluir explícitamente instrucciones como:

- `in $rt, puerto` (para leer datos desde un puerto de E/S).
- `out $rs, puerto` (para escribir datos hacia un puerto de E/S).
- Variantes de estas instrucciones para distintos tamaños de palabra (bytes, medias palabras, palabras de 32 bits).

1.5. ¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?

Colocación de la Dirección:

El procesador coloca la dirección física de 32 bits en el Bus de direcciones y activa las señales del Bus de Control (indica una operación de LECTURA o ESCRITURA).

Decodificación de Dirección:

El Decodificador de Direcciones del sistema evalúa los bits más significativos de la dirección presente en el bus.

Activación de Chip Select (CS):

- Si la dirección cae dentro del rango de la RAM, el decodificador habilita el chip de la RAM.
- Si la dirección coincide con el rango asignado a un periférico (como los registros del sensor de luz o del tensiómetro), el decodificador deshabilita la RAM y activa la línea Chip Select (CS) del controlador del periférico correspondiente.

Transferencia de Datos:

El periférico seleccionado toma el control del Bus de Datos para entregar la lectura (`lw`) o recibir la orden enviada por la CPU (`sw`).

1.6. ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?

¿Es posible?

En sistemas embebidos simples o en simuladores académicos en modo bare-metal (como MARS por defecto), sí es posible. Sin embargo, en un sistema operativo real (como Linux sobre MIPS), un programa en Modo Usuario (unprivileged mode) NO puede acceder directamente a las direcciones del dispositivo.

Mecanismos de Protección:

- Separación de Modos (Kernel vs. User): MIPS utiliza el bit de modo en el registro de estado del Coprocesador 0 (CP0 Status Register) para distinguir entre el modo Kernel (privilegiado) y modo Usuario.
- Unidad de Gestión de Memoria (MMU) y Paginación: La MMU traduce direcciones virtuales a físicas. Las páginas de memoria donde residen los registros MMIO están marcadas con permisos exclusivos para el Kernel.
- Excepciones de Protección: Si un proceso de usuario intenta ejecutar lw o sw hacia una dirección MMIO protegida, la MMU genera una excepción de Violación de Memoria/Acceso no Autorizado (Bus Error / Address Error), cediendo el control al Sistema Operativo para abortar la aplicación.

1.7. ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?

Las esperas activas (o polling) consumen el 100 % de los recursos de procesamiento del CPU en bucles de comprobación continuos. Para evitarlo se emplean:

E/S Dirigida por Interrupciones (Interrupt-Driven I/O):

El procesador envía la orden de inicio al dispositivo (ej. iniciar medición de tensión arterial) y continúa ejecutando otras tareas/procesos.

Cuando el dispositivo finaliza, genera una señal de interrupción de hardware. La CPU interrumpe temporalmente el flujo actual, atiende la rutina de servicio a la interrupción (ISR), lee los datos y retoma la tarea previa.

Acceso Directo a Memoria (DMA - Direct Memory Access):

Un controlador de DMA asume la transferencia masiva de bloques de datos entre el dispositivo y la memoria RAM sinintervención de la CPU. Al finalizar la transferencia completa, el DMA emite una única interrupción al procesador.

1.8. Análisis y Discusión de los Resultados

En el desarrollo de esta práctica se implementaron dos controladores esenciales en lenguaje ensamblador MIPS32 mediante Memory-Mapped I/O:

Controlador del Sensor de Luminosidad:

Se verificó la secuencia de control obligatoria en sistemas de hardware: escribir la palabra de inicio (0x1) en la dirección de LuzControl y realizar una encuesta (polling) sobre LuzEstado. Se constató la importancia del manejo de códigos de error cuando el estado retorna -1 (error de hardware) versus la lectura exitosa del dato entero (0 a 1023) contenido en LuzDatos.

Controlador de Tensión Arterial (controladortension):

Demostró la gestión de periféricos de procesamiento diferido. Tras escribir 1 en TensionControl, el dispositivo requiere un tiempo de cálculo en el que TensionEstado permanece en 0. Una vez habilitado (1), se obtuvieron correctamente los valores de presión sistólica y diastólica y se retornaron en los registros de salida correspondientes (\$v0 y \$v1), respetando estrictamente las convenciones de llamadas de funciones de MIPS.

El ejercicio permitió consolidar que el acceso a dispositivos de hardware a bajo nivel mediante MMIO se abstrae internamente como operaciones convencionales de lectura y escritura en memoria, requiriendo un estricto respeto a las banderas de estado para sincronizar la velocidad de la CPU con la latencia del periférico.